

Programação e Desenvolvimento de Software I

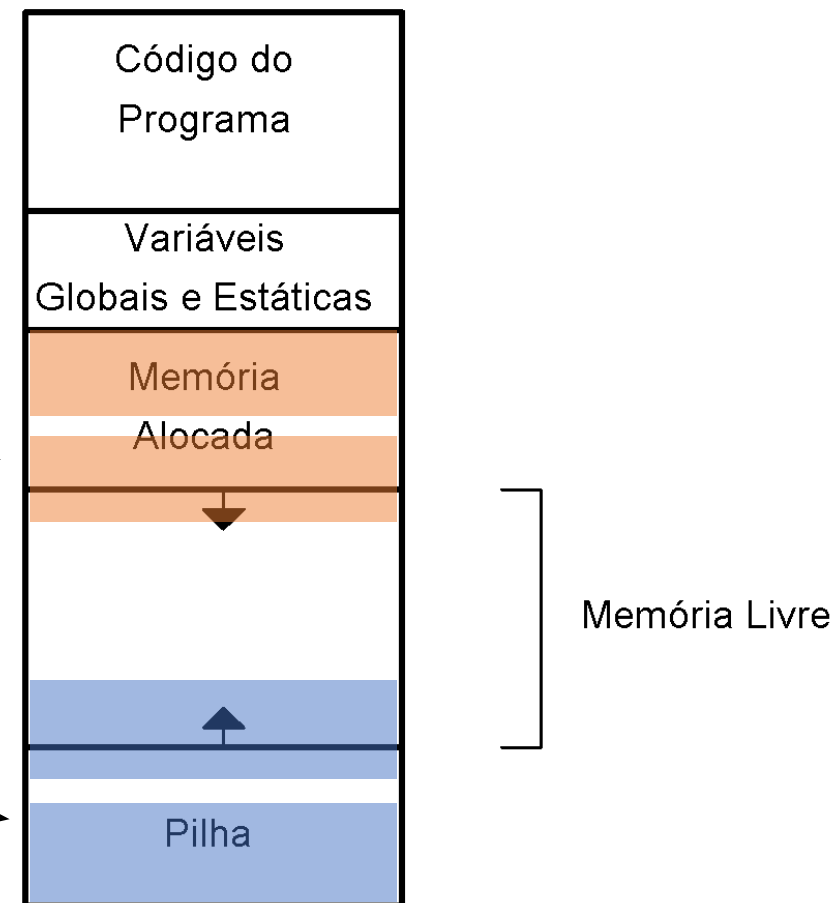
Recursividade

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Pilha de execução

Uso na memória

- Antes de entrar na **recursão** precisamos saber o que é a **pilha de execução**
- Ao executarmos um programa, sabemos que:
 - Quando usamos `malloc`, estamos alocando na **memória dinâmica ou heap**
 - Quando declaramos variáveis normais, elas estão na **pilha ou stack**
- E o que acontece com as funções?
 - Todas as chamadas ficam alocadas na pilha!



Esquema da memória do sistema

Pilha de execução

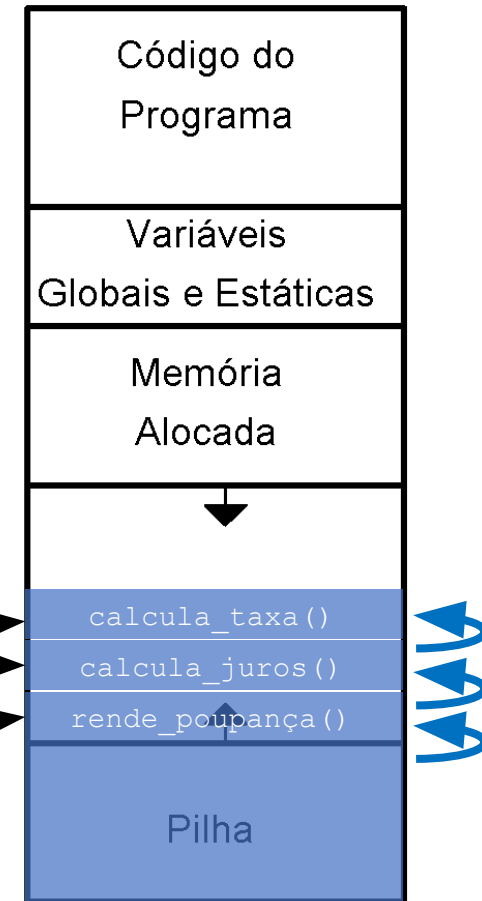
Uso na memória

```
float calcula_juros(float saldo) {  
    float taxa = 0.1;  
    return saldo * taxa;  
}
```

```
float calcula_taxa(float saldo) {  
    if (saldo < 1000) {  
        return 0.0;  
    }  
    return 3.14;  
}
```

```
void rende_poupanca(float *saldo) {  
    *saldo = *saldo +  
        calcula_juros(*saldo) - calcula_taxa(*saldo);  
}
```

```
int main() {  
    float poupanca = 1500.0;  
    rende_poupanca(&poupanca); // rendimento apos o primeiro mes  
    printf("Tenho R$ %.2f na poupanca\n", poupanca);  
    return 0;  
}
```



Esquema da memória do sistema

Pilha de execução

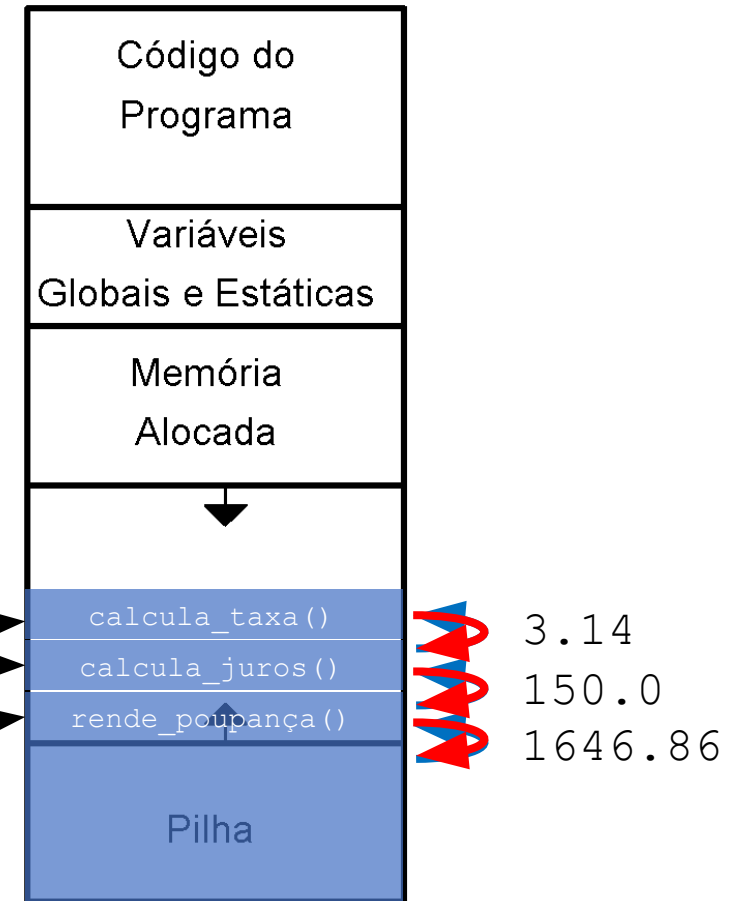
Uso na memória

```
float calcula_juros(float saldo) {  
    float taxa = 0.1;  
    return saldo * taxa;  
}
```

```
float calcula_taxa(float saldo) {  
    if (saldo < 1000) {  
        return 0.0;  
    }  
    return 3.14;  
}
```

```
void rende_poupanca(float *saldo) {  
    *saldo = *saldo +  
        calcula_juros(*saldo) - calcula_taxa(*saldo);  
}
```

```
int main() {  
    float poupanca = 1500.0;  
    rende_poupanca(&poupanca); // rendimento apos o primeiro mes  
    printf("Tenho R$ %.2f na poupanca\n", poupanca);  
    return 0;  
}
```



Esquema da memória do sistema

Pilha de execução

Uso na memória

- Lembrando os limites de memória:
 - **Heap:**
 - Geralmente, o limite é a memória disponível
 - **Stack:**
 - É limitada, depende do compilador, arquitetura, SO, etc.
 - Geralmente no range de 1–8MB
- O que vai acontecer com o nosso programa se preenchemos a pilha de funções até estourar a memória? 🤔



Esquema da memória do sistema

Recursão

O que é?

- A **recursividade** é uma forma interessante de resolver problemas:
 - Por meio da divisão dos problemas em problemas menores de **mesma natureza**
 - Se a **natureza dos subproblemas** é a mesma do problema, **o mesmo método** usado para reduzir o problema pode ser usado para reduzir os subproblemas e assim por diante
- Chamamos de **recursividade** ou **recursão** quando uma função chama a si mesma!
 - A cada chamada é criada na **memória uma nova ocorrência da função** com comandos e variáveis “isolados” das ocorrências anteriores

```
int hellow(int a) {  
    if (a == 0) {  
        return 1;  
    }  
    return hellow(a - 1);  
}
```

Recursão

Vantagens e desvantagens

■ Vantagens

- Torna a escrita do código mais **simples e elegante**
- Fácil de entender e de manter



■ Desvantagens

- Quando o loop recursivo é muito grande é consumida muita memória nas chamadas a diversos níveis de recursão
 - Cada chamada recursiva aloca memória para os parâmetros e variáveis
- Em muitos casos uma soluções iterativas são mais rápidas e gastam menos memória



Pilha de execução

Exemplo simples de recursão

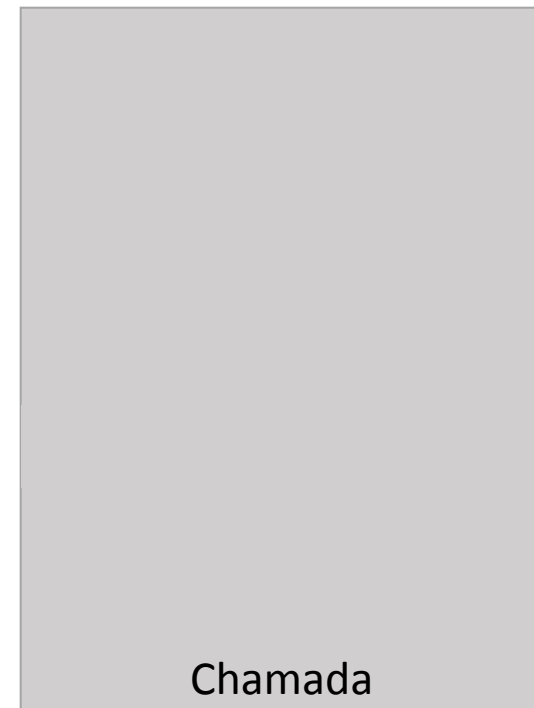
- Funções recursivas:
 - Esse programa tem algum problema?
 - Sim! Ele **não para...**
 - Bom, ele para quando **toda a memória acabar!**

```
void foo(int n) {  
    printf("%d\n", n);  
    foo(n - 1);  
}
```

```
int main() {  
    foo(4);  
    return 0;  
}
```

- **Como podemos melhorar isso?**

Pilha de execução

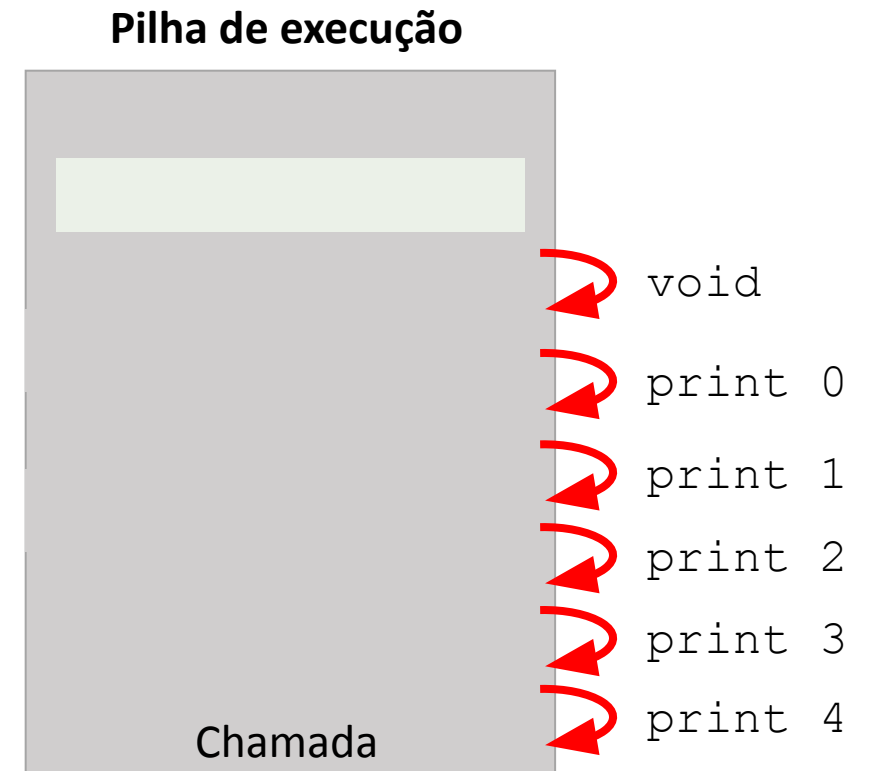


Pilha de execução

Exemplo simples de recursão

- Definir um **caso trivial** ou **condição de parada!**

```
void foo(int n) {  
    if (n < 0)  
        return;  
  
    foo(n - 1);  
    printf("%d\n", n);  
}  
  
int main() {  
    foo(4);  
    return 0;  
}
```



Recursividade

Caso base e chamadas recursivas

- Toda recursividade é composta por um caso base e pelas chamadas recursivas:

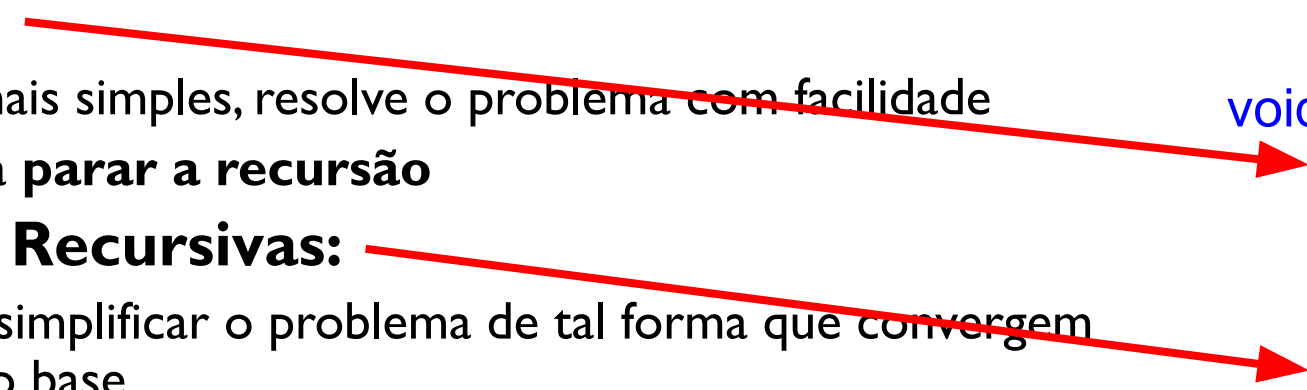
- **Caso base:**

- É o caso mais simples, resolve o problema com facilidade
 - Usada para **parar a recursão**

- **Chamadas Recursivas:**

- Procuram simplificar o problema de tal forma que convergem para o caso base
 - Reduzir o problema a um ou mais problemas menores (subproblemas) de mesma natureza

```
void foo(int n) {  
    if (n < 0)  
        return;  
  
    foo(n - 1);  
    printf("%d\n", n);  
}
```



- **Exemplo:**

- Fatorial de um número e o n-ésimo termo da sequência de Fibonacci

Recursividade

Função que soma todos os números entre 0 e n

- Como podemos modificar esse código para que **some todos números até n** ?

```
void foo(int n) {  
    if (n < 0)  
        return;  
  
    foo(n - 1);  
    printf("%d\n", n);  
}  
  
int main() {  
    foo(4);  
    return 0;  
}
```

Parâmetros de entrada
e de saída

Condição de parada

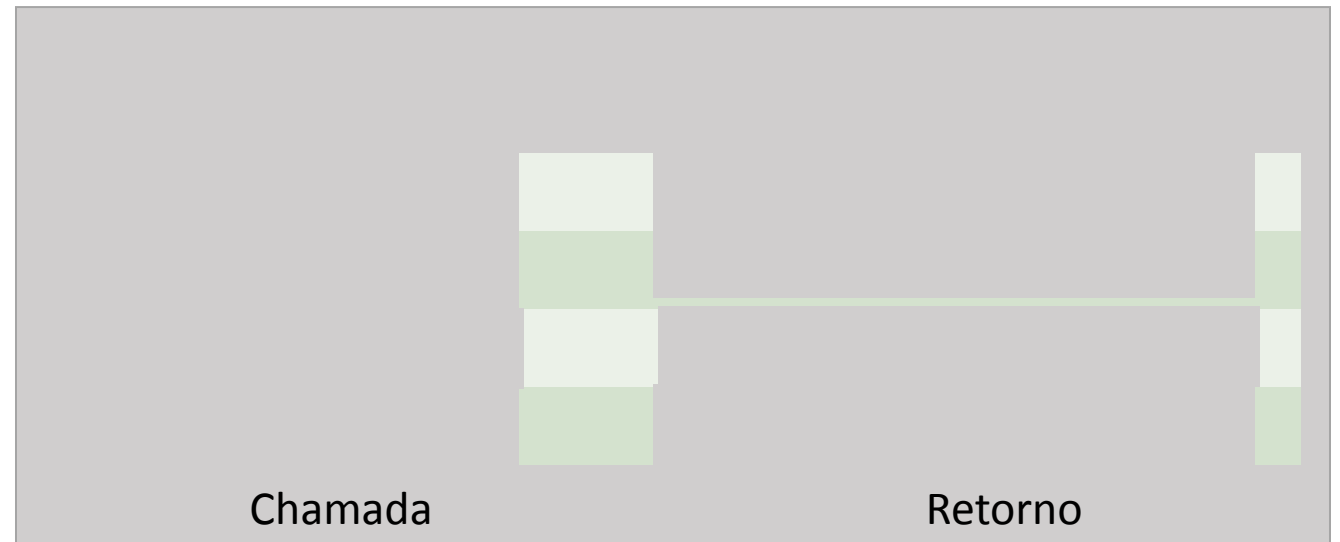
Chamada recursiva

Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

Pilha de execução



Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

Pilha de execução

Chamada	Retorno
soma (1)	-> return 1+0
soma (2)	-> return 2+soma (1)
soma (3)	-> return 3+soma (2)

Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

Pilha de execução

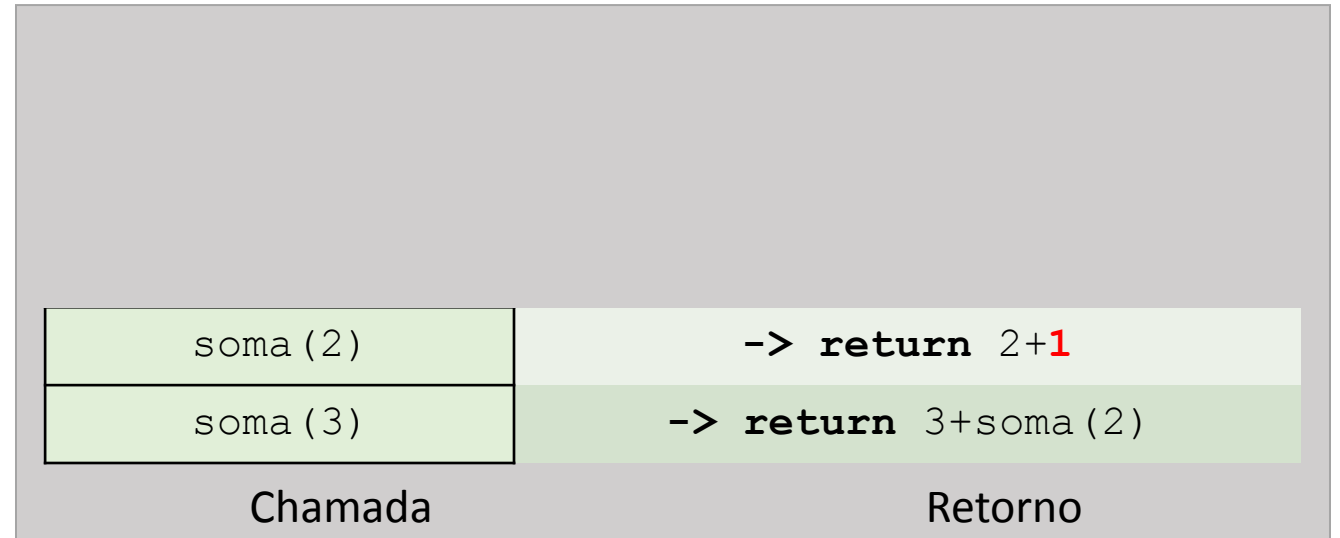
Chamada	Retorno
soma (1)	-> return 1+0
soma (2)	-> return 2+soma (1)
soma (3)	-> return 3+soma (2)

Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

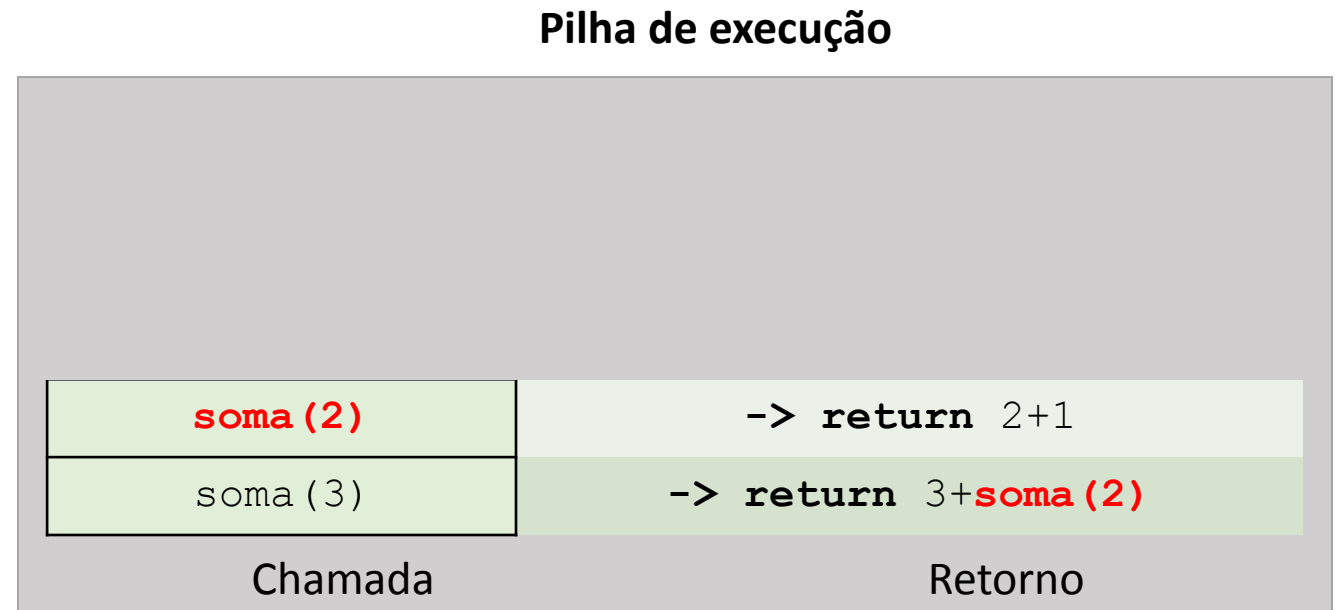
Pilha de execução



Função que soma todos os números entre 0 e n

```
int soma(int n) {
    if (n == 0) {
        return 0;
    }
    return n + soma(n - 1);
}

int main() {
    printf("Soma: %d\n", soma(3));
    return 0;
}
```

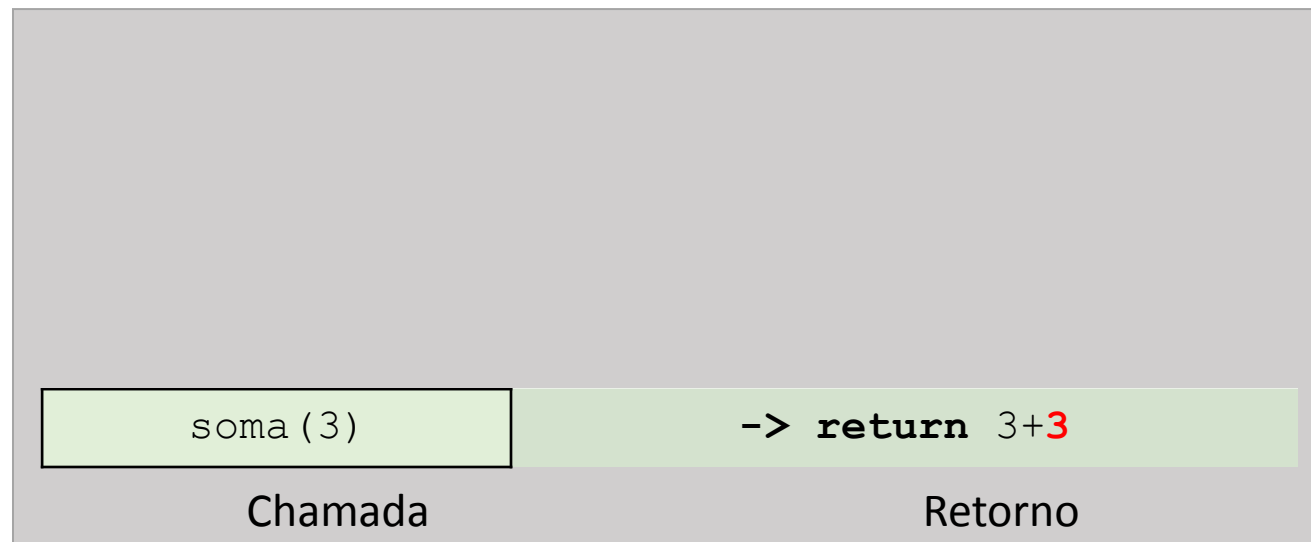


Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

Pilha de execução

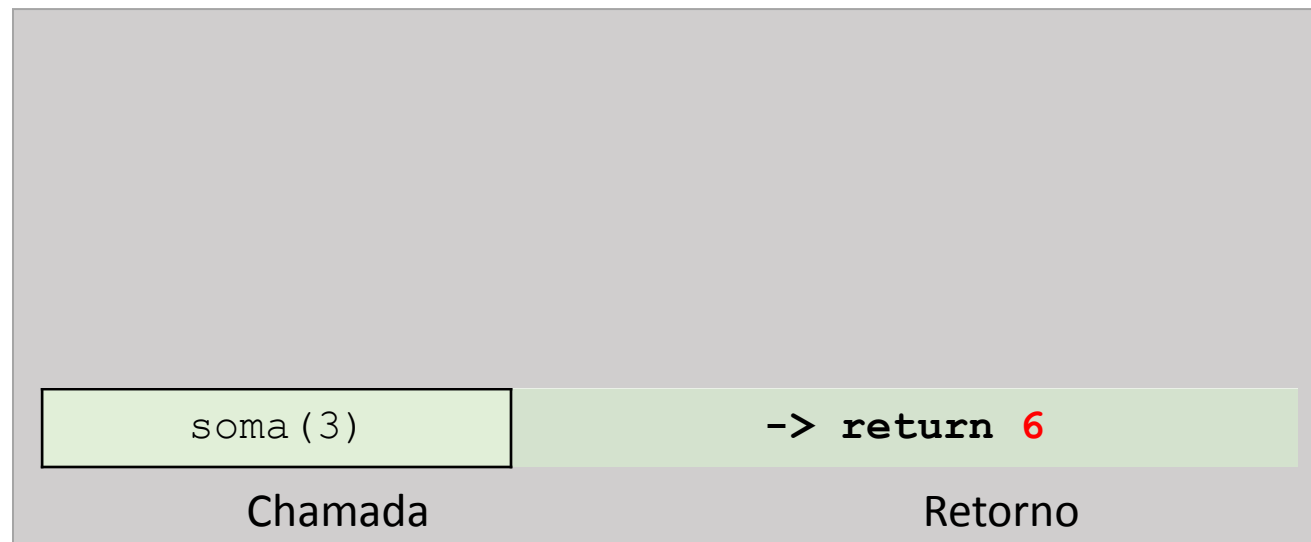


Recursividade

Função que soma todos os números entre 0 e n

```
int soma(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    return n + soma(n - 1);  
}  
  
int main() {  
    printf("Soma: %d\n", soma(3));  
    return 0;  
}
```

Pilha de execução



Função fatorial

- A função **fatorial** de um inteiro não negativo pode ser definida como:

$$n! = \begin{cases} 1 & n = 0 \\ n \times (n-1)! & n > 0 \end{cases}$$

- Esta definição estabelece um **processo recursivo** para calcular o fatorial de um inteiro n
- **Caso trivial:** $n=0$
 - Neste caso: $n! = 1$
- **Método geral:** $n * (n-1) !$

Função fatorial

- Assim, usando-se este processo recursivo, o cálculo de $4!$:

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

$$\begin{aligned} 4! &= 4 * 3! \\ &= 4 * (3 * 2!) \\ &= 4 * (3 * (2 * 1!)) \\ &= 4 * (3 * (2 * (1 * 0!))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * 6 \\ &= 24 \end{aligned}$$

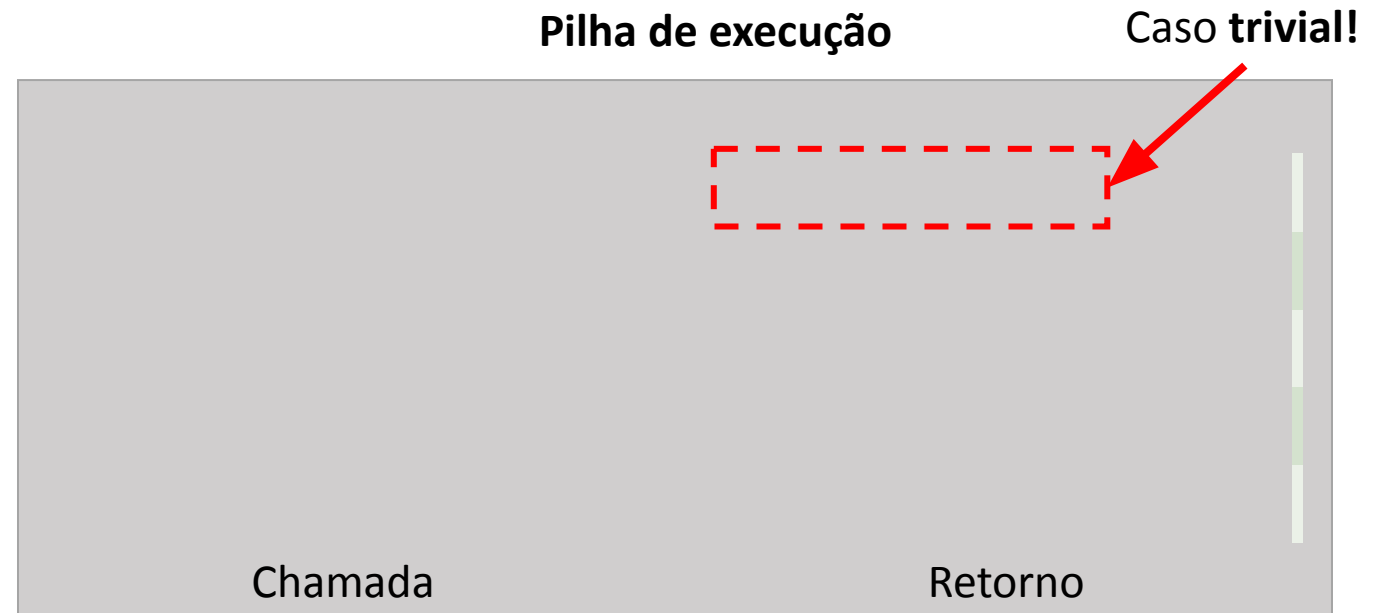
- Verifiquemos o funcionamento da função recursiva na **Pilha de Execução!**

Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```



Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução

fatorial(0)	-> return 1
fatorial(1)	-> return 1*fatorial(0)
fatorial(2)	-> return 2*fatorial(1)
fatorial(3)	-> return 3*fatorial(2)
fatorial(4)	-> return 4*fatorial(3)
Chamada	Retorno

Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução

fatorial(0)	-> return 1
fatorial(1)	-> return 1*1
fatorial(2)	-> return 2*fatorial(1)
fatorial(3)	-> return 3*fatorial(2)
fatorial(4)	-> return 4*fatorial(3)
Chamada	Retorno

Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução

Pilha de execução	
Chamada	Retorno
fatorial(1)	-> return 1*1
fatorial(2)	-> return 2*fatorial(1)
fatorial(3)	-> return 3*fatorial(2)
fatorial(4)	-> return 4*fatorial(3)

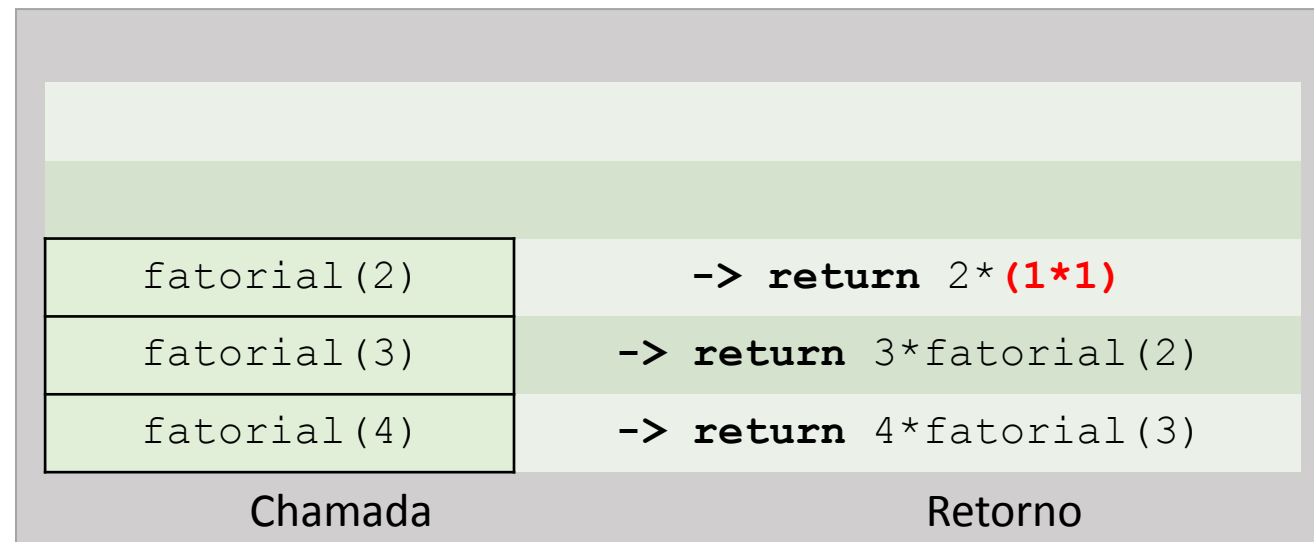
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



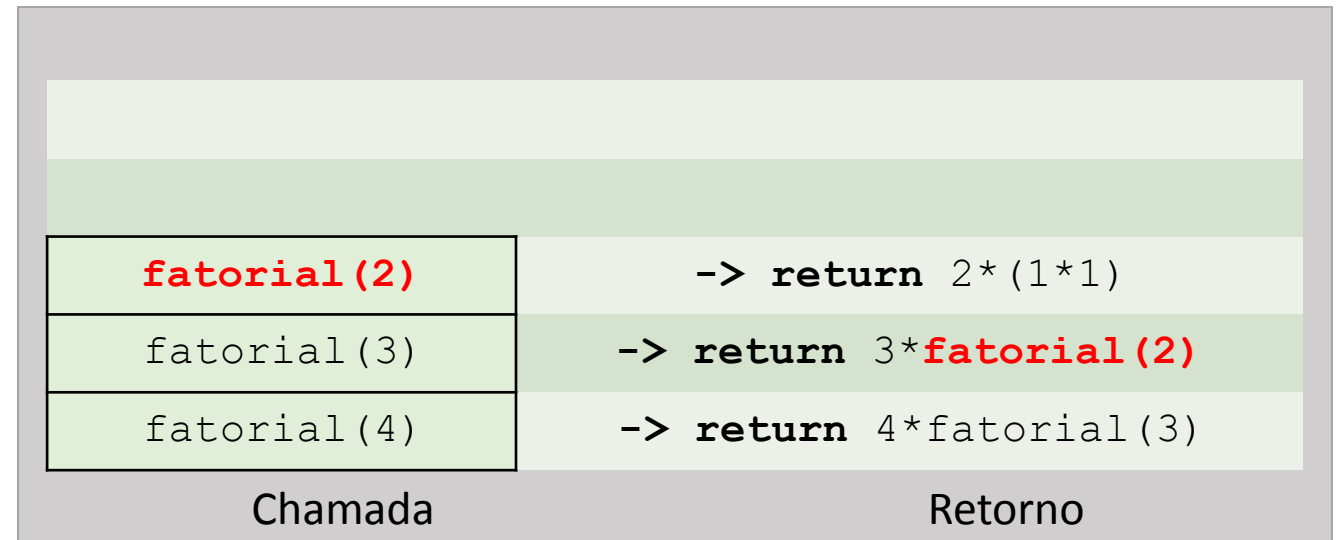
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



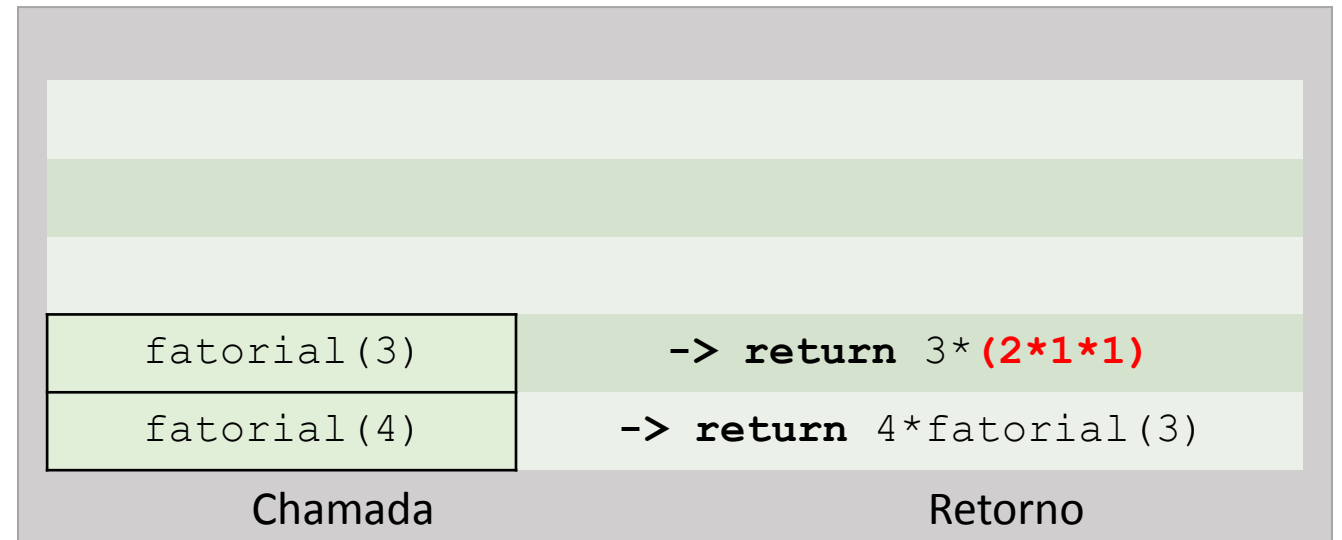
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



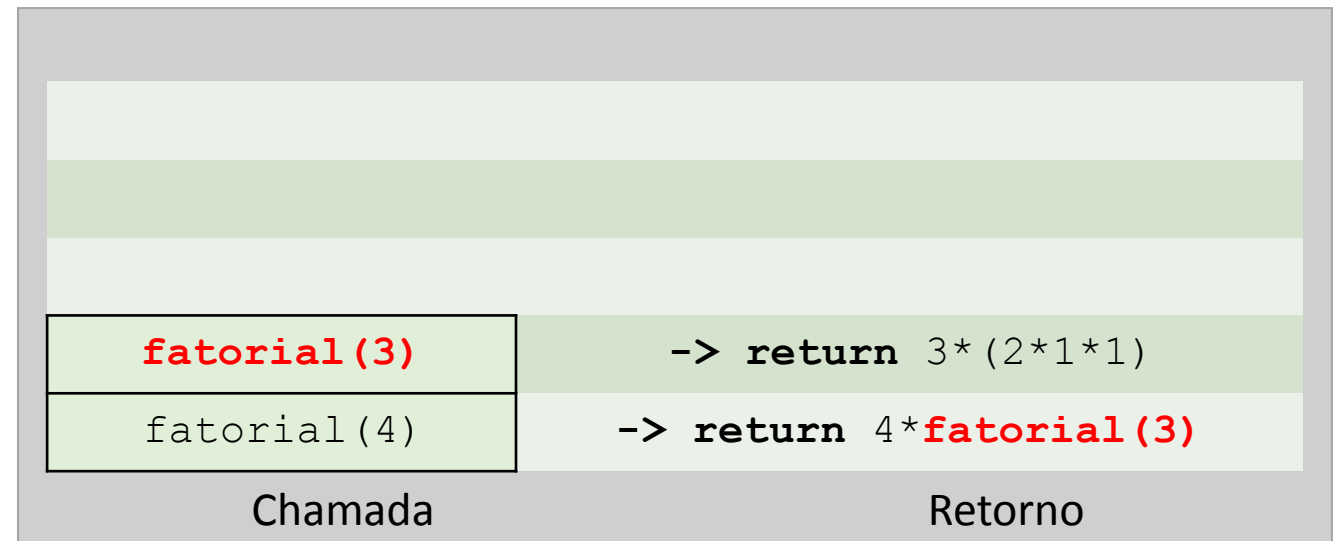
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



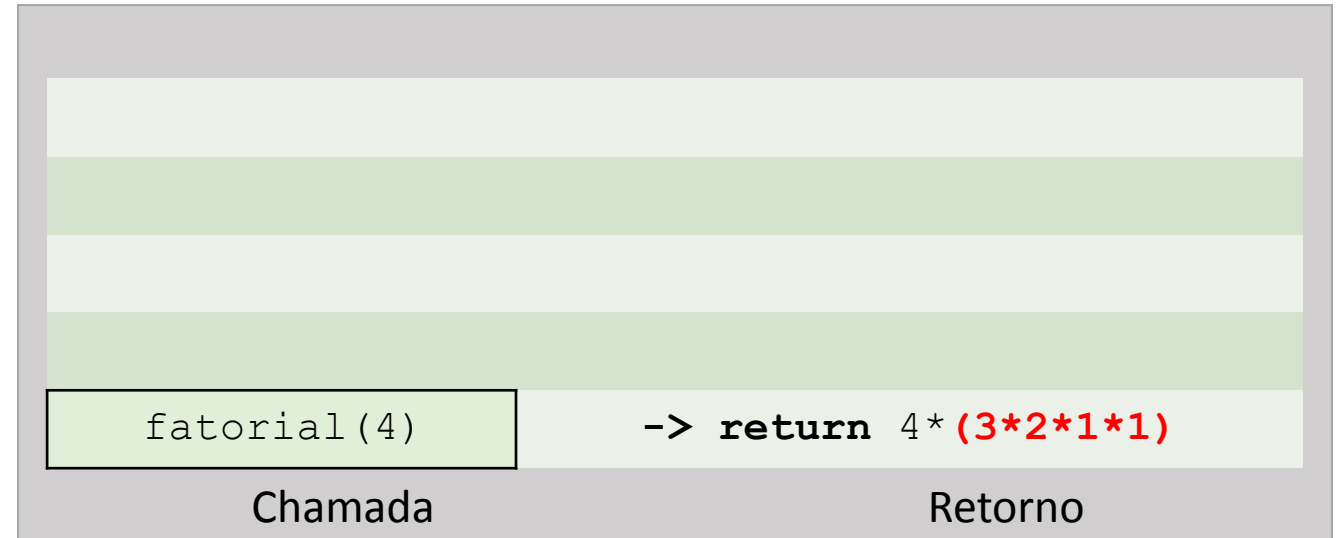
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



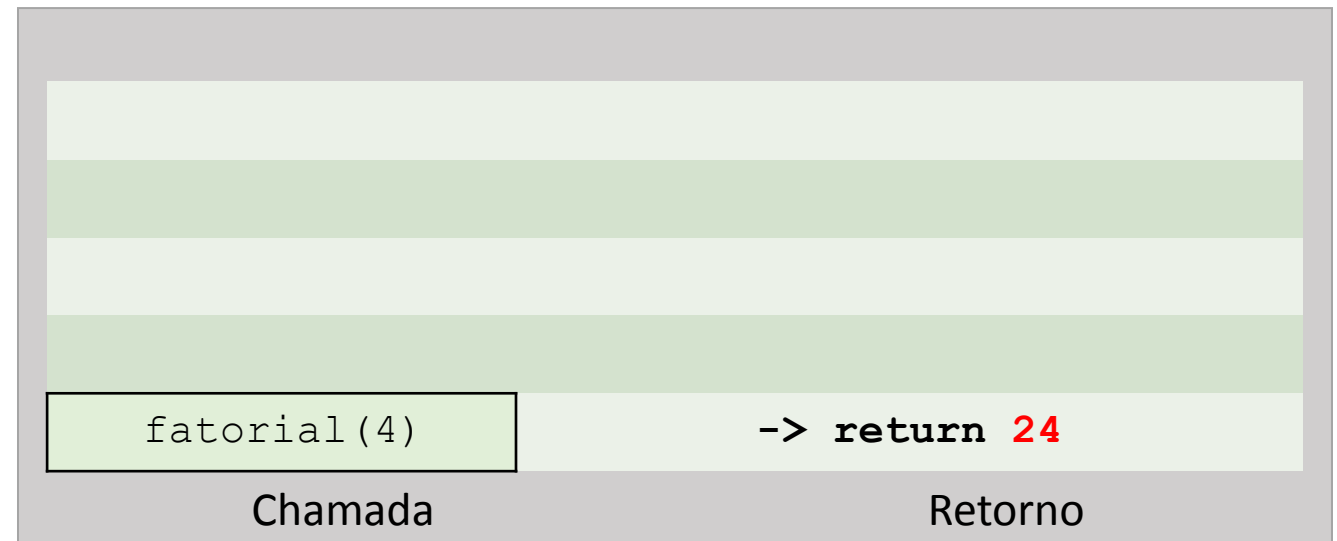
Função fatorial

Passo a passo

- Consideremos o cálculo de $4!$
 - Agora o processo inverso “desempilhar”

```
int fatorial(int n) {  
    if (n == 0) return 1;  
    return n * fatorial(n - 1);  
}
```

Pilha de execução



Função Fibonacci

- A função Fibonacci retorna o n -ésimo número da sequência:
 - $1, 1, 2, 3, 5, 8, 13, \dots$
- Os dois primeiros termos são iguais a 1 e cada um dos demais números é a soma dos dois números imediatamente anteriores
 - Sendo assim, o n -ésimo número $fib(n)$ é dado por:

$$fib(n) = \begin{cases} 1 & n = 1 \\ 1 & n = 2 \\ fib(n-2) + fib(n-1) & n > 2 \end{cases}$$

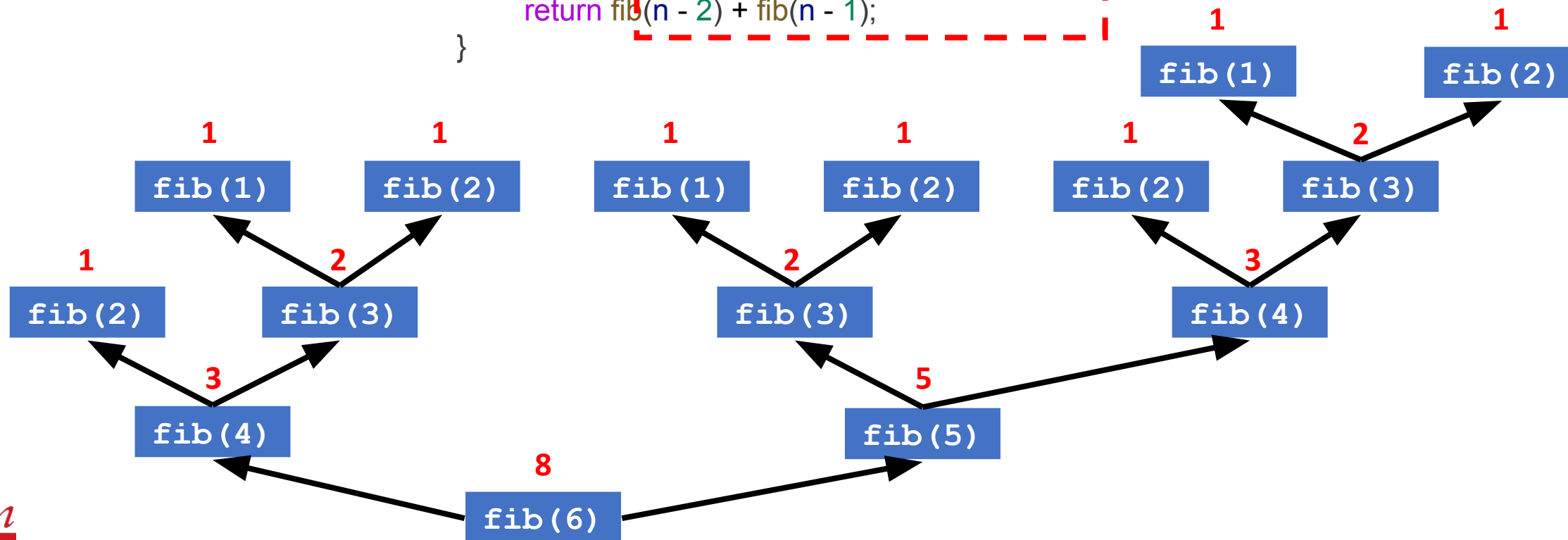
Função Fibonacci

Implementação recursiva

- Veja uma implementação recursiva para esta função:

```
int fib(int n) {  
    if (n <= 2) {  
        return n;  
    }  
    return fib(n - 2) + fib(n - 1);  
}
```

Repare na quantidade de chamadas!
Para cada número > 2 são realizadas duas chamadas



Função Fibonacci

Implementação iterativa

- A função recursiva para cálculo do n-ésimo termo da sequência é extremamente ineficiente, uma vez que **recalcula o mesmo valor várias vezes**



```
int fib_iterative(int n) {  
    if (n <= 1) {  
        return n;  
    }  
    int a = 0, b = 1, c;  
    for (int i = 2; i <= n; i++) {  
        c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

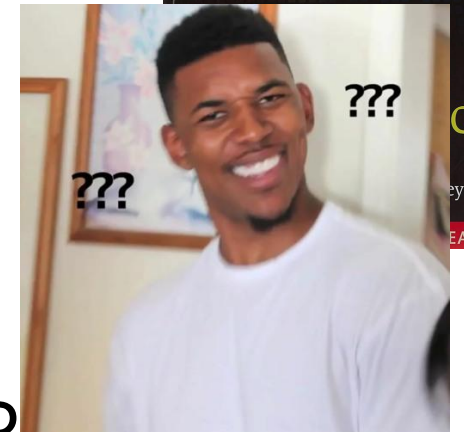
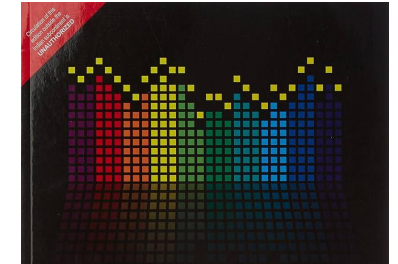
Função Fibonacci

Análise de tempo de execução

- O livro dos autores Brassard e Bradley (*Fundamentals of Algorithmics*, 1996, pág. 73) apresenta um quadro comparativo de tempos de execução das versões iterativa e recursiva...

Entrada N	10	20	30	50	100
Recursivo					
Iterativo					

- Um algoritmo recursivo nem sempre é o melhor caminho para se resolver um problema
- No entanto, a recursividade muitas vezes torna o **algoritmo mais simples e elegante**



Função Fibonacci

Similar ao triângulo de pascal...

- Se lembram desta pergunta?
 - Tem uma logica relativamente parecida...

$$C(n, r) = \frac{C(n, r-1) * (n-r+1)}{r}$$

Parte 2 – Triângulo de Pascal (10 pontos)

O Triângulo de Pascal é formado por coeficientes binomiais e seus elementos podem ser calculados facilmente quando organizados em um triângulo. Veja um Triângulo de Pascal com 6 linhas:

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1

Note que em cada linha (exceto a primeira):

1. O primeiro e o último elemento são sempre iguais a 1
2. Os demais elementos são dados pela soma de dois elementos na linha anterior. Mais precisamente, o i -ésimo elemento na linha L é dado pela soma do i -ésimo elemento da linha $(L-1)$ e seu antecessor. Por exemplo, o primeiro elemento 10 na última linha ($i = 3$) é o resultado da soma de 6 ($i = 3$) e 4 ($i = 2$) na linha anterior. O segundo elemento 10 na última linha ($i = 4$) é o resultado da soma de 4 ($i = 4$) e 6 ($i = 3$) na linha anterior.

Triângulo de pascal

Exemplo recursivo

```
int triangulo(int row, int col) {
    if (col == 0 || col == row) {
        return 1;
    } else if (col == 1 || (col + 1) == row) {
        return row;
    } else {
        return triangulo(row - 1, col - 1) + triangulo(row - 1, col);
    }
}

int main() {
    int n = 5; // linhas do triângulo de Pascal
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++) {
            printf("%d ", triangulo(i, j));
        }
        printf("\n");
    }
    return 0;
}
```

Recursão

Problema de ordenação

- Implemente uma função que classifique os elementos de um vetor em ordem crescente usando o algoritmo **quicksort**
 - Embora seja **recursivo**, o **quicksort** é um dos **melhores algoritmos de ordenação**!
- **Algoritmo:**
 1. Seja **m** o elemento na posição **central** no vetor
 2. Seja **i** o índice do primeiro e **j**, o índice do último elemento do vetor
 3. Enquanto **i** for menor ou igual a **j**, faça com que:
 - a) O valor de **i** cresça até encontrar um elemento maior que **m**
 - b) O valor de **j** diminua até encontrar um elemento menor que **m**
 - c) Haja a troca entre os elementos que ocupam as posições **i** e **j**

Recursão

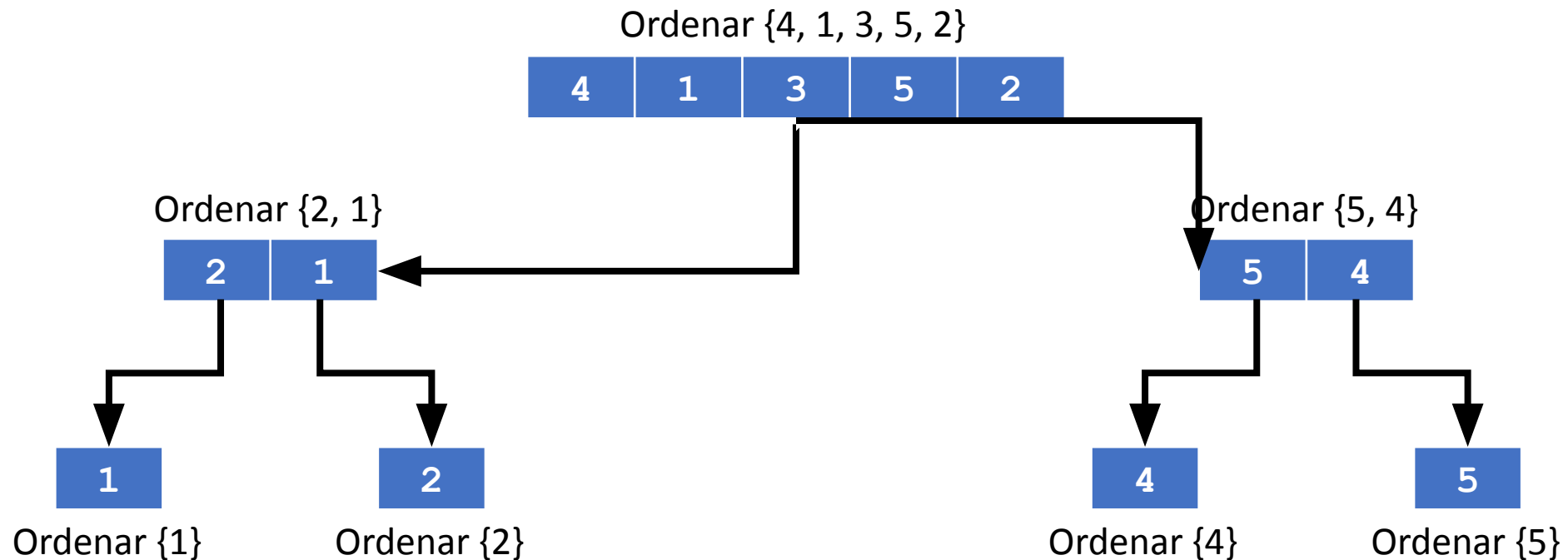
Problema de ordenação

- Ao final desses passos, a situação do vetor será a seguinte:
 - à esquerda da posição central, existem somente elementos menores do que **m**
 - à direita da posição central, existem somente elementos maiores do que **m**
- Assim, o problema de ordenar o vetor se reduz ao problema de ordenar cada uma dessas “metades”
- Os mesmos passos serão aplicadas repetidas vezes à cada nova “metade”, até que cada metade contenha um único elemento (caso trivial)

Recursão

Problema de ordenação

- Considere o exemplo:
 - Como a natureza dos problemas é sempre a mesma (“ordenar um vetor”)
 - O mesmo método pode ser usado para cada subproblema!



Recursão

Quicksort

- A função de recursão do quicksort

```
void quicksort(int v[], int low, int high) {  
    int i = low;  
    int j = high;  
    int tmp = 0;  
    int pivot = v[(i + j) / 2];  
  
    do {  
        while (v[i] < pivot) i++;  
        while (v[j] > pivot) j--;  
        if (i <= j) {  
            tmp = v[i];  
            v[i] = v[j];  
            v[j] = tmp;  
            i++;  
            j--;  
        }  
    } while (i <= j);  
  
    if (low < j)  
        quicksort(v, low, j);  
    if (high > i)  
        quicksort(v, i, high);  
}
```


Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r